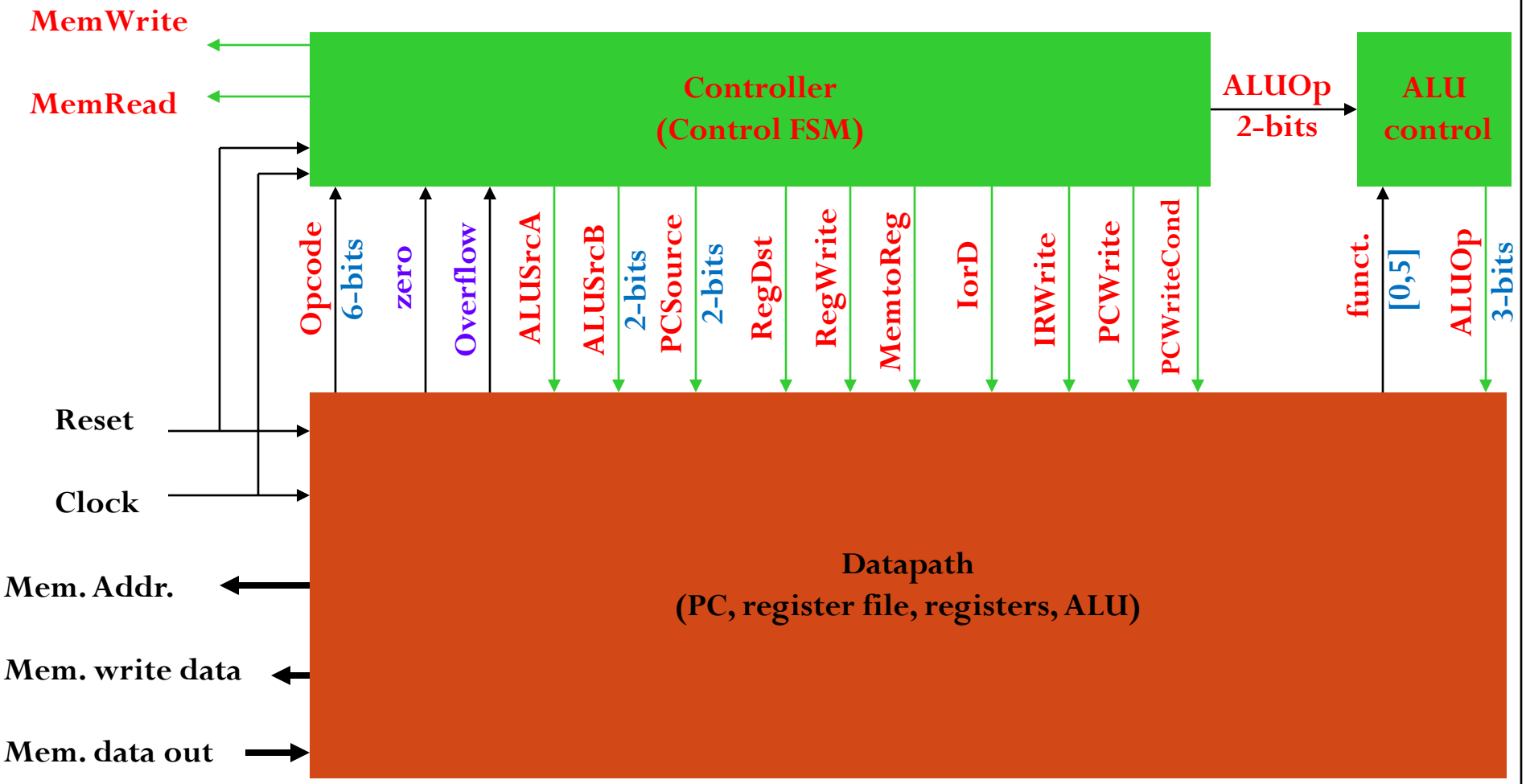
Computer System Architecture

Parallelism (Microarchitecture level)

Pipelining

Block Diagram of a Processor



ILP: Instruction Level Parallelism

- Single-cycle and multi-cycle datapaths execute one instruction at a time.
- How can we get better performance?
- Answer: Execute multiple instructions at a time:
  - Pipelining – Enhance a multi-cycle datapath to fetch one instruction every cycle.
  - Parallelism/Multiple Issue – Fetch multiple instructions every cycle.

Pipelining in a μP

- Break each instruction down into a set of tasks so that instructions can be executed in a staggered fashion.
- Divide datapath into nearly equal tasks, to be performed serially and requiring non-overlapping resources.
- Insert registers at task boundaries in the datapath; registers pass the output data from one task as input data to the next task.
- Synchronize tasks with a clock having a cycle time just enough to complete the longest task.

Stages of Execution in Pipelined MIPS

5 stage instruction pipeline

1) I-fetch: Fetch Instruction, Increment PC

2) Decode: Instruction, Read Registers

3) Execute:

   Mem-reference: Calculate Address

   R-format: Perform ALU Operation

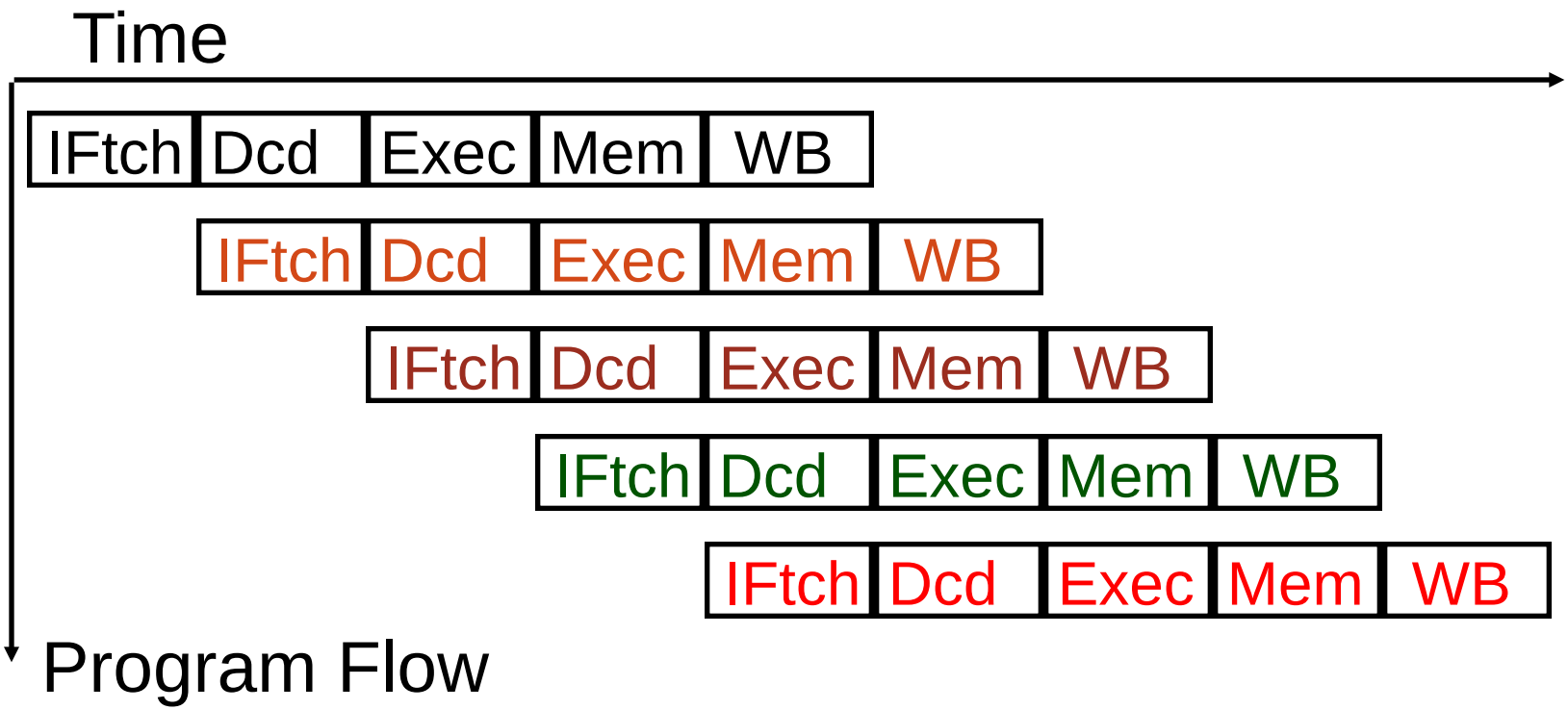
4) Memory:

   Load: Read Data from Data Memory

   Store: Write Data to Data Memory

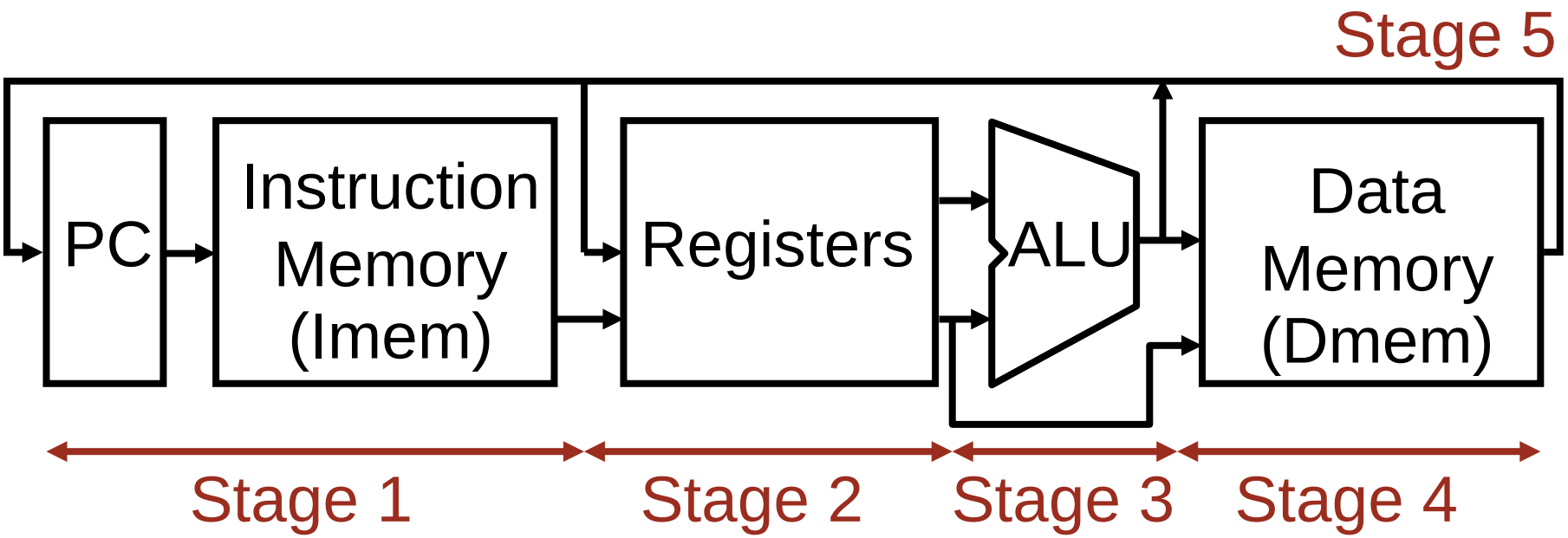
5) Write Back: Write Data to Register

Pipelined Execution Representation

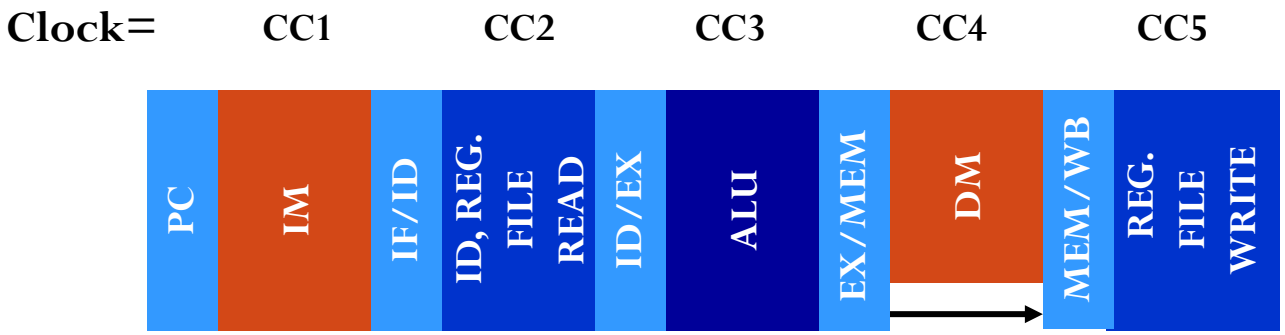


- To simplify pipeline, every instruction takes same number of steps, called stages (simplification cost ??)
- One clock cycle per stage

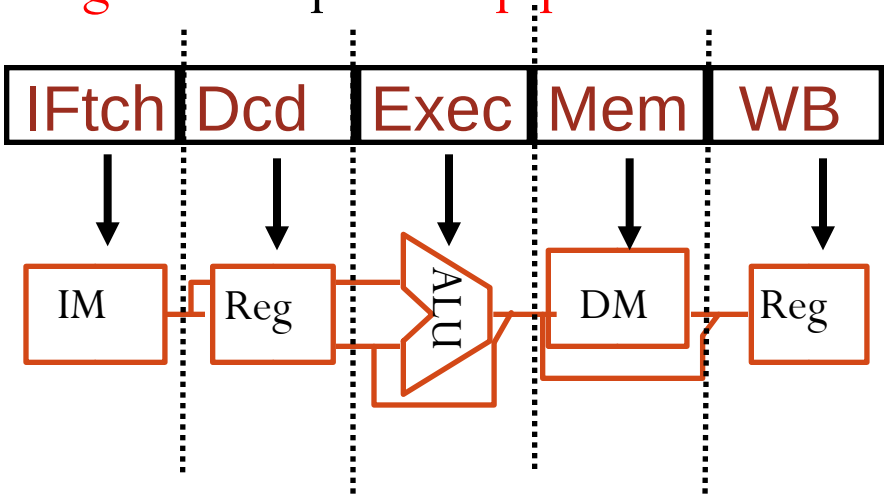
Review: Single-cycle Datapath for MIPS



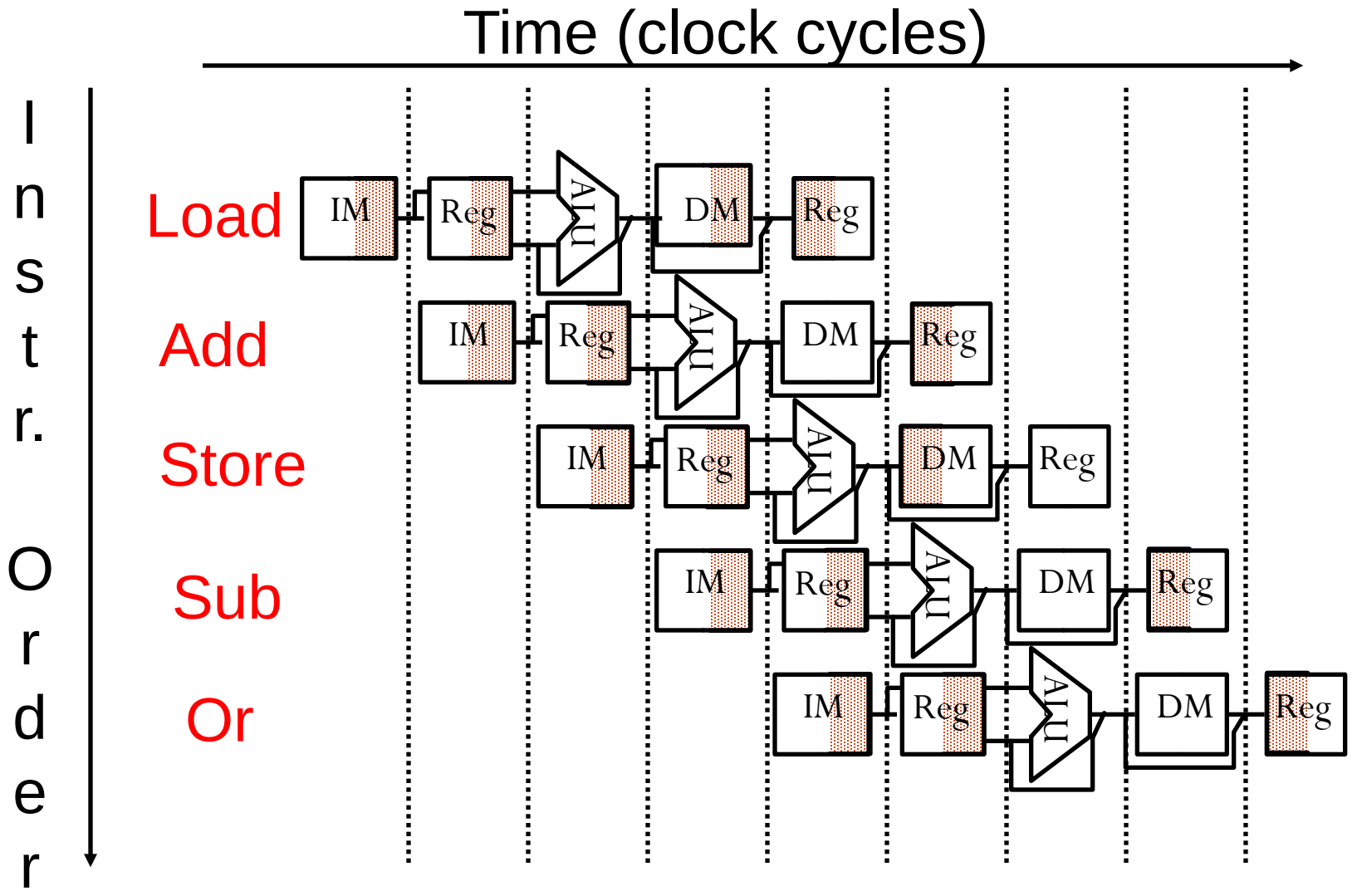
Five-Cycle Pipeline



- Use datapath figure to represent pipeline

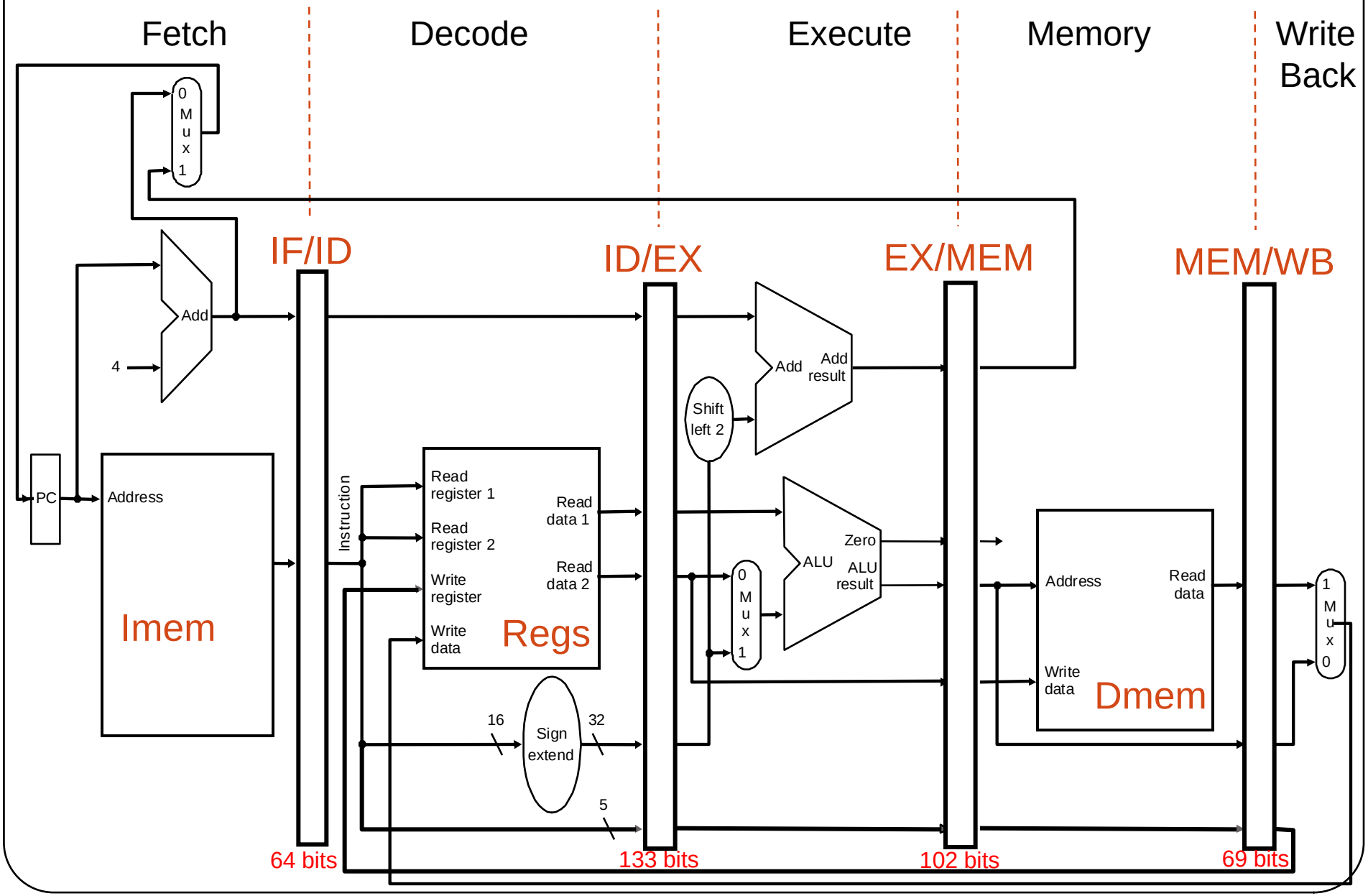


Graphical Pipeline Representation



(right half highlighted means read, left half write)

Pipelined Datapath (with Pipeline Regs)



Required Changes to Datapath

- Introduce registers to separate 5 stages by putting IF/ID, ID/EX, EX/MEM, and MEM/WB registers in the datapath.
- Next PC value is computed in the 3rd step, but we need to bring in next instn in the next cycle – Move PCSrc Mux to 1st stage
- Branch address is computed in 3rd stage. With pipeline, the PC value has changed! Must carry the PC value along with instn. Width of IF/ID register = (IR)+(PC) = 64 bits.
- For lw instn, we need write register address at stage 5. But the IR is now occupied by another instn! So, we must carry the IR destination field as we move along the stages. See connection in fig. Length od ID/EX register = (Reg1)+(Reg2)+(offset)+(PC)+ destn = 133 bits

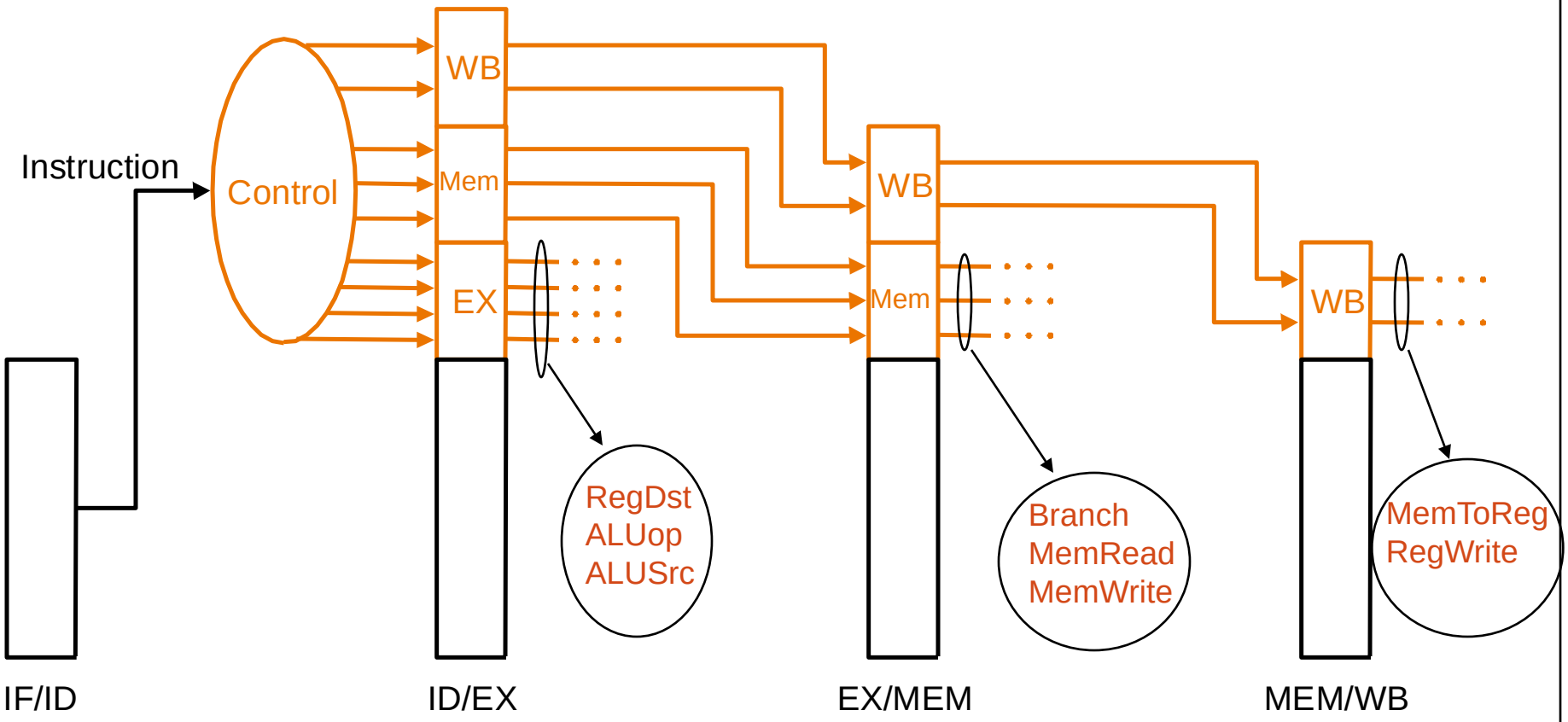
Pipeline Register Functions

- Four pipeline registers are added:

Register name	Data held
IF/ID	PC+4, Instruction word (IW)
ID/EX	PC+4, R1, R2, IW(0-15) sign ext., IW(11-15)
EX/MEM	PC+4, zero, ALUResult, R2, IW(11-15) or IW(16-20)
MEM/WB	M[ALUResult], ALUResult, IW(11-15) or IW(16-20)

Pipelined Control

- Start with single-cycle controller
- Group control lines by pipeline stage needed
- Extend pipeline registers with control bits



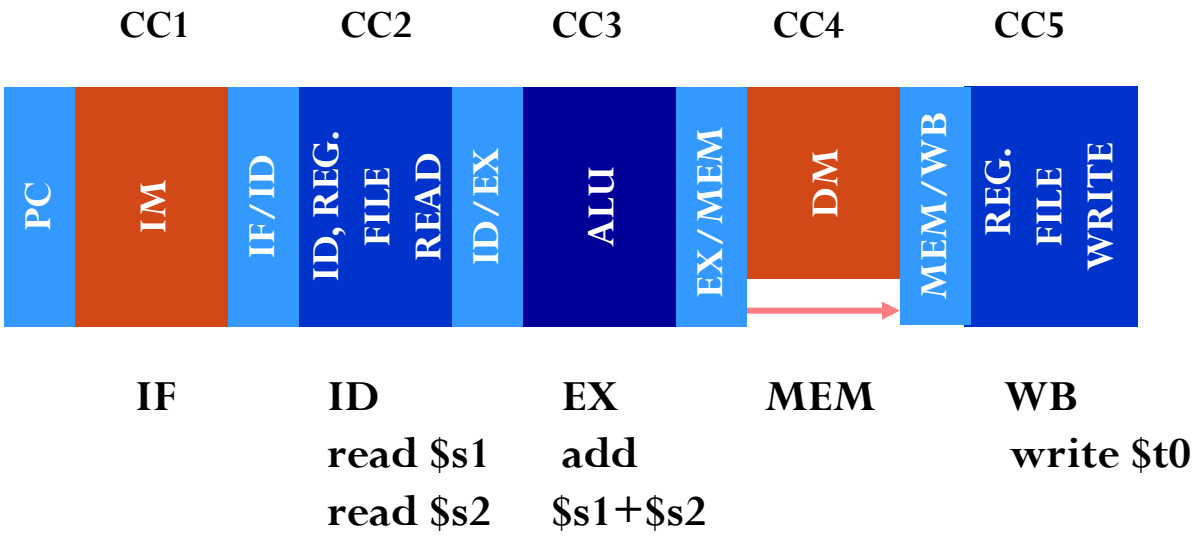
Add Instruction

- add $t0, $s1, $s2

  Machine instruction word

  000000 10001 10010 01000 00000 100000

  opcode $s1 $s2 $t0 function

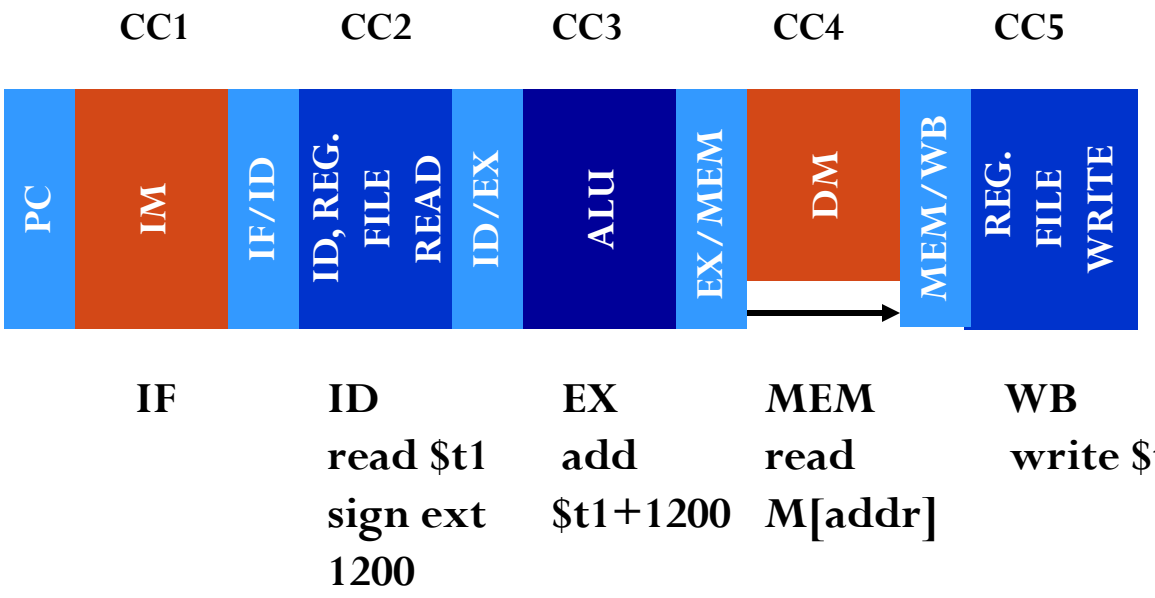


Load Instruction

- lw $t0, 1200 ($t1)

100011 01001 01000 0000 0100 1000 0000

opcode $t1 $t0 1200

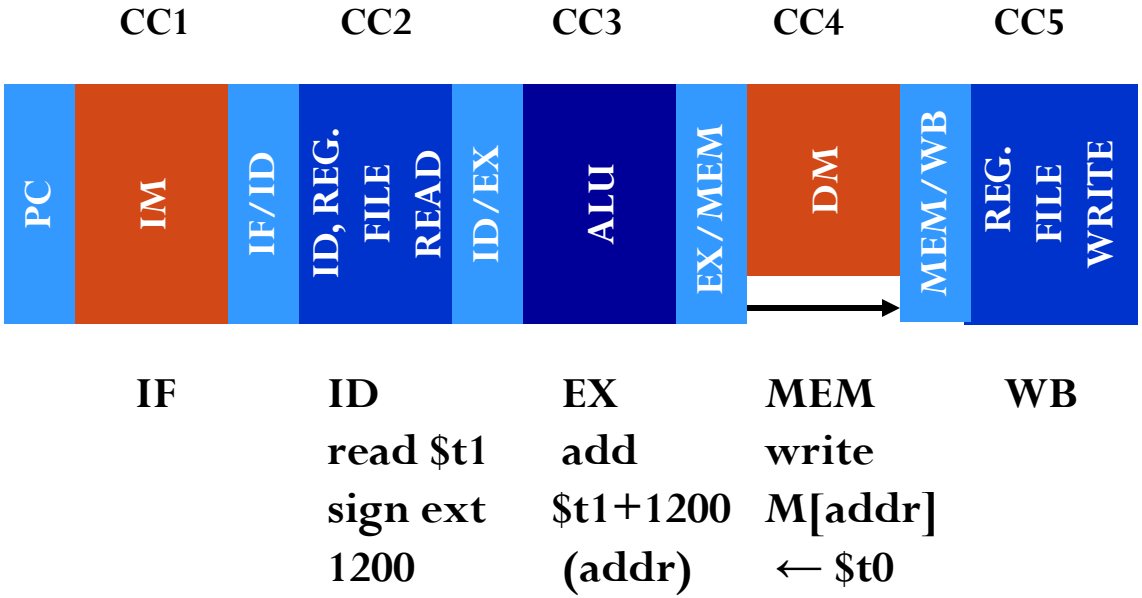


Store Instruction

- sw $t0, 1200 ($t1)

101011 01001 01000 0000 0100 1000 0000

opcode $t 0 $t1 1200

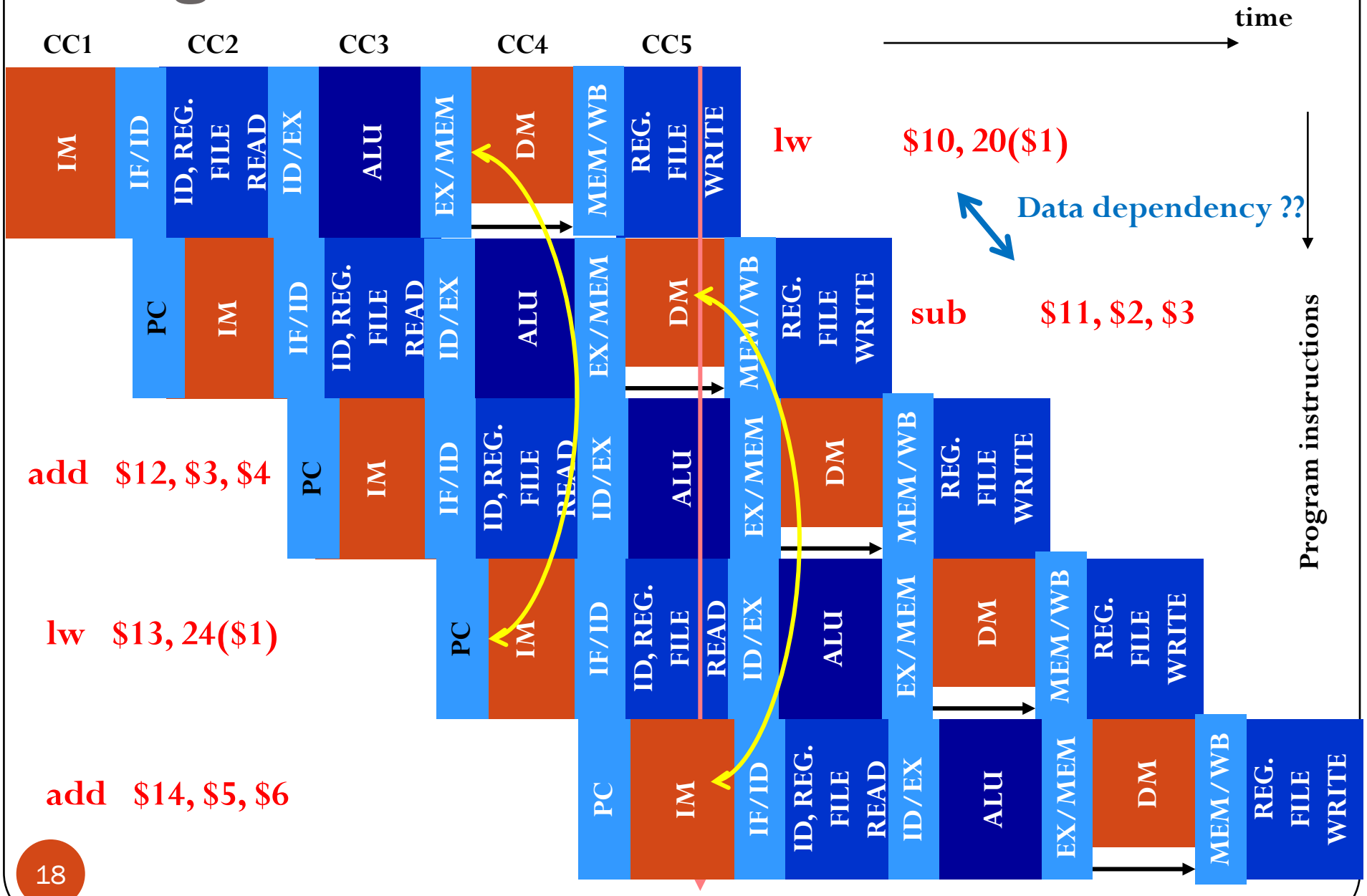


Executing a Program using Pipeline

Consider a five-instruction segment:

```
lw    $10, 20($1)
sub   $11, $2, $3
add   $12, $3, $4
lw    $13, 24($1)
add   $14, $5, $6
```

Program Execution



Performance

- Average cycles per instruction (CPI)
- Cycle time (clock period, T)
- Execution time of a program
- For single-cycle datapath:
  - CPI = 1
  - T = hardware time for lw instruction
  - Execution time of a program

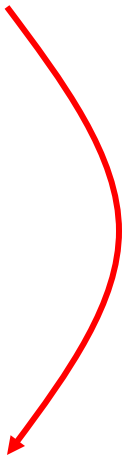
    = T × CPI × (Total instructions executed)

How Long Does It Take in Datapath

- Assume control logic is fast and does not affect the critical timing. Major time components are ALU, memory read/write, and register read/write.
- Time for hardware operations, suppose
  - Memory read or write 2ns
  - Register read 1ns
  - ALU operation 2ns
  - Register write 1ns

Single-Cycle Datapath

- R-type 6ns
- Load word (I-type) 8ns
- Store word (I-type) 7ns
- Branch on equal (I-type) 5ns
- Jump (J-type) 5ns
- Clock cycle time = 8ns
- Each instruction takes *one* cycle



Single-Cycle Datapath

Instruction class	Instr. fetch (IF)	Instr. Decode (also reg. file read) (ID)	Execution (ALU Operation) (EX)	Data access (MEM)	Write Back (Reg. file write) (WB)	Total time
lw	2ns	1ns	2ns	2ns	1ns	8ns
sw	2ns	1ns	2ns	2ns		8ns
R-format add, sub, and, or, slt	2ns	1ns	2ns		1ns	8ns
B-format, beq	2ns	1ns	2ns			8ns

No operation on data; idle times equalize instruction lengths.

Multicycle Datapath (fully utilize Clock)

- Clock cycle time is determined by the longest operation, ALU or memory:
  - Clock cycle time = 2ns
- Cycles per instruction:
  - lw 5 (10ns)
  - sw 4 (8ns)
  - R-type 4 (8ns)
  - beq 3 (6ns)
  - j 3 (6ns)

CPI of a Multicycle Computer

$$\text{CPI} = \frac{\sum_k (\text{Instructions of type k}) \times \text{CPI}_k}{\sum_k (\text{instructions of type k})}$$

where

CPI_k = Cycles for instruction of type k

Note: CPI is dependent on the instruction mix of the program being run. Standard benchmark programs are used for specifying the performance of CPUs.

Example

- Consider a program containing:
  - loads 25%
  - stores 10%
  - branches 11%
  - jumps 2%
  - Arithmetic 52%
- CPI = 0.25×5 + 0.10×4 + 0.11×3 + 0.02×3 + 0.52×4

  = 4.12 for multicycle datapath
- CPI = 1.00 for single-cycle datapath

Multicycle vs. Single-Cycle

$$\text{Performance ratio} = \text{Single cycle time / Multicycle time}$$

$$= \frac{\text{(CPI} \times \text{cycle time) for single-cycle}}{\text{(CPI} \times \text{cycle time) for multicycle}}$$

$$= \frac{1.00 \times 8\text{ns}}{4.12 \times 2\text{ns}} = 0.97$$

Single cycle is faster in this case, but is it always so?

Consider Another Example

- For this program:
  - loads 5%
  - stores 5%
  - branches 30%
  - jumps 10%
  - Arithmetic 50%
- CPI = 0.05×5 + 0.05×4 + 0.30×3 + 0.10×3 + 0.50×4

  = 3.65 for multicycle datapath
- CPI = 1.00 for single-cycle datapath

Multicycle vs. Single-Cycle

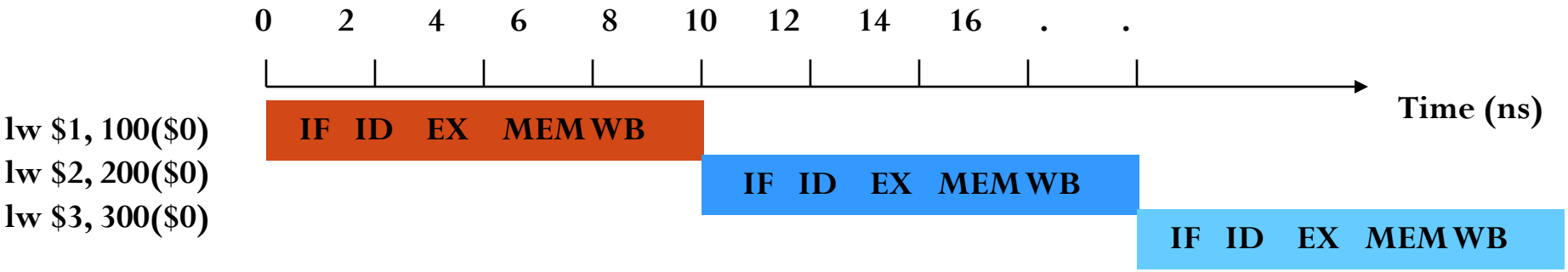
$$\text{Performance ratio} = \text{Single cycle time / Multicycle time}$$

$$= \frac{\text{(CPI} \times \text{cycle time) for single-cycle}}{\text{(CPI} \times \text{cycle time) for multicycle}}$$

$$= \frac{1.00 \times 8\text{ns}}{3.65 \times 2\text{ns}} = 1.096$$

Multicycle is faster in this case, so the performance ratio depends on the instruction mix. ***Can we do better?***

Execution Time: Single-Cycle



Clock cycle time = 8 ns

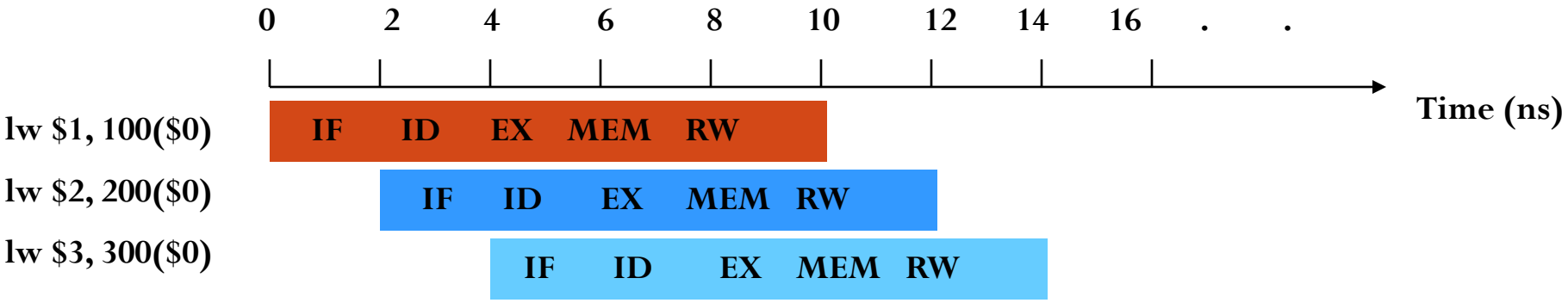
Total time for executing three lw instructions = 24 ns

Pipelined Datapath (fully utilize Resources)

Instruction class	Instr. fetch (IF)	Instr. Decode (also reg. file read) (ID)	Execu-tion (ALU Opera-tion) (EX)	Data access (MEM)	Write Back (Reg. file write) (WB)	Total time
lw	2ns	~~1ns~~ 2ns	2ns	2ns	~~1ns~~ 2ns	10ns
sw	2ns	~~1ns~~ 2ns	2ns	2ns	~~1ns~~ 2ns	10ns
R-format: add, sub, and, or, slt	2ns	~~1ns~~ 2ns	2ns	2ns	~~1ns~~ 2ns	10ns
B-format: beq	2ns	~~1ns~~ 2ns	2ns	2ns	~~1ns~~ 2ns	10ns

No operation on data; idle time inserted to equalize instruction lengths.

Execution Time: Pipeline



Clock cycle time = 2 ns, *four times faster than single-cycle clock*

Total time for executing three lw instructions = 14 ns

$$\text{Performance ratio} = \frac{\text{Single-cycle time}}{\text{Pipeline time}} = \frac{24}{14} = 1.7$$

Pipeline Performance (Long Progm)

Clock cycle time = 2 ns

1,003 *lw instructions:*

Total time for executing 1,003 lw instructions = 2,014 ns

$$\text{Performance ratio} = \frac{\text{Single-cycle time}}{\text{Pipeline time}} = \frac{8{,}024}{2{,}014} = 3.98$$

10,003 *lw instructions:*

Performance ratio = 80,024 / 20,014 = 3.998

Clock cycle ratio (8ns/ 2ns = 4)

Pipeline performance approaches clock-cycle ratio for long programs.

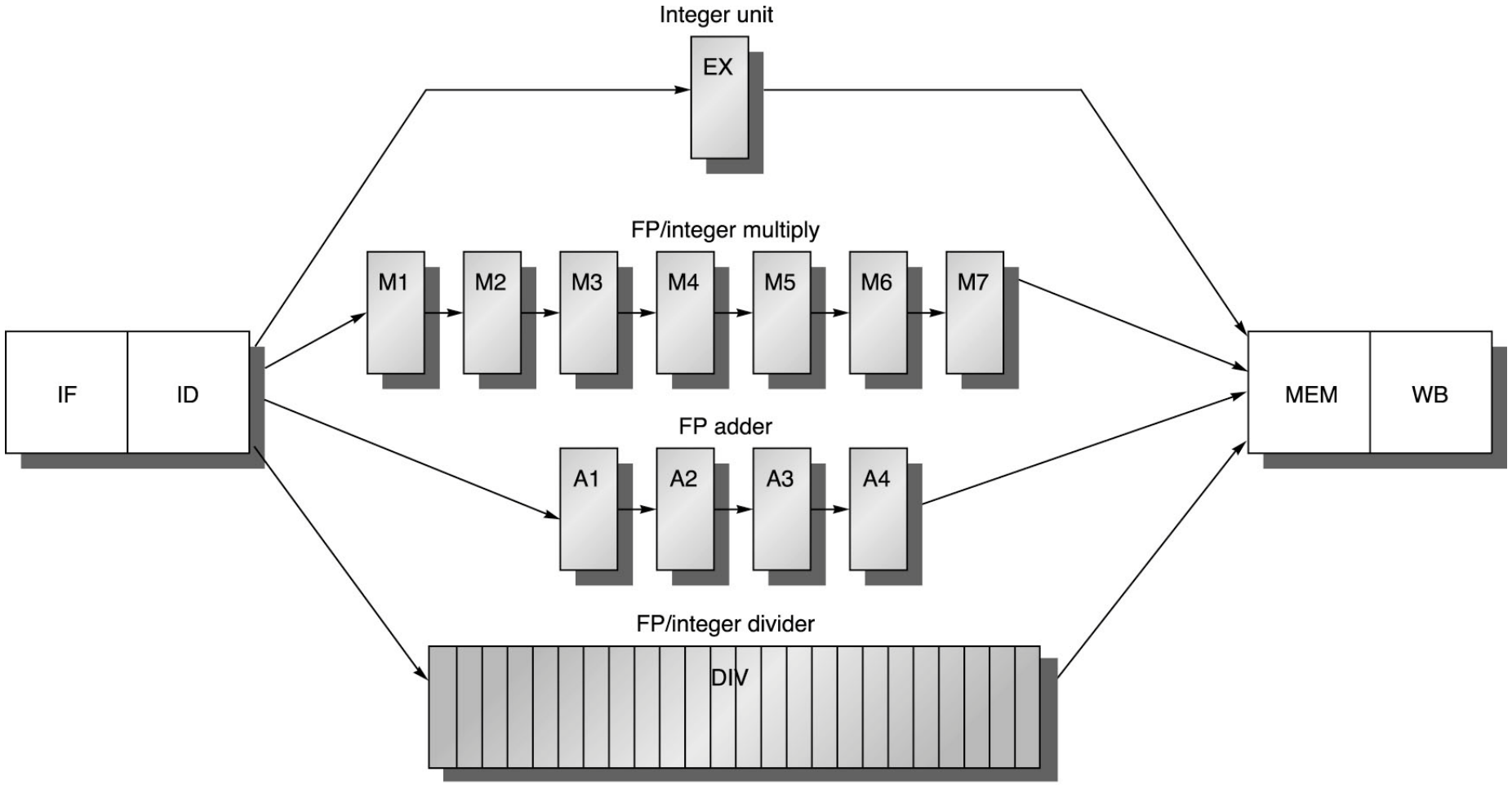
Advantages of Pipeline

- After the initial latency of five cycles (CC5), one instruction is completed each cycle; CPI ≈ 1.
  - *Pipeline latency is defined as the number of stages in the pipeline, or*
  - *The number of clock cycles after which the first instruction is completed.*
- The clock cycle time is about four times shorter than that of single-cycle datapath and about the same as that of multicycle datapath.
- But for multicycle datapath, CPI = 3., ….
- So, pipelined execution is faster, but . . .

Problems for Pipelining

- Hazards prevent next instruction from executing during its designated clock cycle, limiting speedup
  - Structural hazards: HW cannot support this combination of instructions (single person to fold and put clothes away)
  - Control hazards: conditional branches & other instructions may stall the pipeline delaying later instructions (must check detergent level before washing next load)
  - Data hazards: Instruction depends on result of prior instruction still in the pipeline (matching socks in later load)

Real MIPS R4000 pipeline



Assignment - 3

- Assume datapath delays
  - 200 ps for memory access
  - 100 ps for ALU operation
  - 50 ps for register file read or write
- Consider SPECINT2000* (benchmark) instruction mix:
  - 25% lw
  - 10% sw
  - 11% branch
  - 2% jump
  - 52% ALU instr.
- For pipeline implementation consider same instruction mix while:
  - Neglect initial latency (reasonable for long programs).
  - One instruction completed every clock cycle unless delayed by hazard as:
    - lw 2 cycles in 50% cases due to hazard
    - branch 2 cycles in 25% cases due to hazard

Assignment - 3

- Calculate clock rate and Instruction cycle time for Single-Cycle
- Calculate clock rate, average CPI and average Instruction cycle time for Multi-Cycle
- Calculate clock rate, average CPI and average Instruction cycle time for Pipeline μ-architecture of MPI-light ISA.